Задание #3. Структуры данных: стек, очередь.

Задание #3.1. Структура данных: стек.

1. Для определения абстрактного типа данных стек используйте абстрактный шаблонный класс.

```
template <typename T>
class Stack
{
public:
  virtual ~Stack() {}
  virtual void push(const T& e) = 0; // Добавление элемента
  virtual T    pop() = 0;            // Удаление элемента
  virtual bool isEmpty() = 0;        // Проверка на пустоту
};
```

2. В соответствии с назначенным вариантом задания реализуйте одну из структур данных (шаблонный класс).

   1) СД стек -массив: StackArray – производный от Stack, реализуется через массив, при переполнении инициируется исключение
   2) СД стек-вектор: StackVector – производный от Stack реализуется через массив, при переполнении расширяется (увеличивается в два раза)
   3) СД стек- список: StackList – производный от Stack, реализуется через список (односвязный)

Создайте классы StackOverflow и StackUnderflow для работы с исключительными ситуациями, которые могут возникнуть при работе со стеком, а также класс WrongStackSize для работы с исключительной ситуацией, которая может возникнуть, если в конструкторе стека -массив, неправильно задан размер стека.

Выполните тестирование созданного класса.

3. В соответствии с назначенным вариантом задания реализуйте одну из функций.

   1) Анализ правильности расстановки скобок.

   Функция должна возвращать True, если количество открывающих и закрывающих скобок одного типа совпадает, и они имеют правильную вложенность. Допускаются три вида скобок: круглые, квадратные и фигурные.

   Прототип функции:
   `bool checkBalanceBrackets (const char* text, const int maxDeep);`
   `text` - анализируемый текст, содержащий скобки
   `maxDeep` - максимально возможный уровень вложенности скобок

2) Перевод арифметического выражения из инфиксной формы в постфиксную (обратную польскую запись).

Функция должна формировать выражение в постфиксной форме и генерировать исключение, если возникает ошибка (например, при переполнении стека, неправильной последовательности операндов и операций, ...)
Считать, что выражение состоит из операндов (***одноразрядных*** десятичных чисел), знаков ***бинарных*** операций (+, -, *, /) и ***круглых*** скобок.
Перевод выполнить за ***один проход*** по выражению, в процессе выполнения которого проверить правильность операндов, допустимость операций, правильность скобок.

Прототипы функции
для стека-массив:

```
void getPostfixFromInfix (const char* infix, char* postfix,
                          const size_t stackSize);
```

для стека-вектор, стека-список:

```
void getPostfixFromInfix (const char* infix, char* postfix)
```

`infix` - выражение в инфиксной форме
`postfix` - выражение в постфиксной форме
`stackSize` - максимальный размер стека операций

Задание #3.2. Структура данных: очередь.

1. Для определения абстрактного типа данных очередь используйте абстрактный шаблонный классы.

```
template <typename T>
class Queue
{
public:
  virtual ~Queue () {}
  virtual void enQueue(const T& e) = 0; // Добавление элемента
  virtual T    deQueue() = 0;           // Удаление элемента
  virtual bool isEmpty() = 0;    // Проверка очереди на пустоту
};
```

2. В соответствии с назначенным вариантом задания реализуйте одну из структур данных (шаблонный класс).

   1) СД очередь-кольцо: QueueRing – производный от Queue, реализуется через «кольцевой» массив, при переполнении инициируется исключение
   2) СД очередь-вектор: QueueVector – производный от Queue реализуется через кольцевой массив, при переполнении расширяется (увеличивается в два раза)
   3) СД очередь-список: QueueList – производный от Queue, реализуется через список (односвязный)

Создайте классы QueueOverflow и QueueUnderflow для работы с исключительными ситуациями, которые могут возникнуть при работе с очередью, а также класс

WrongQueueSize для работы с исключительной ситуацией, которая может возникнуть, если в конструкторе «кольцевой» или «ограниченной» очереди, неправильно задан размер очереди.

Выполните тестирование созданного класса.

Варианты заданий #3.1 и #3.2

Вариант	Класс 2 балла	Функция 1 балл	Класс 2 балла
1	StackArray	`getPostfixFromInfix`	QueueList
2	StackVector	`checkBalanceBrackets`	QueueRing
3	StackList	`getPostfixFromInfix`	QueueVector
4	StackArray	`checkBalanceBrackets`	QueueVector
5	StackVector	`getPostfixFromInfix`	QueueList
6	StackList	`checkBalanceBrackets`	QueueRing

По выполненному заданию необходимо пройти собеседование
Проект загрузить в VPL «Задание #3: Стек. Очередь».